Lifelong Mode Mapping

Lifelong Mode Mapping

1. Course Content
2. Function Introduction
3. Case Demonstration
   3.1 Saving the Pose Graph
   3.2 Saving the Grid Map
4. Restoring the Mapping State
   4.1 Restoring from the Mapping Origin (recommended)
   4.2 Restoring from Any Point in the Map
   4.3 Archiving Mapping Progress

1. Course Content

- Run the example program to build a grid map and a pose map in posegraph format, and use the pose map to restore the previous mapping state and map again

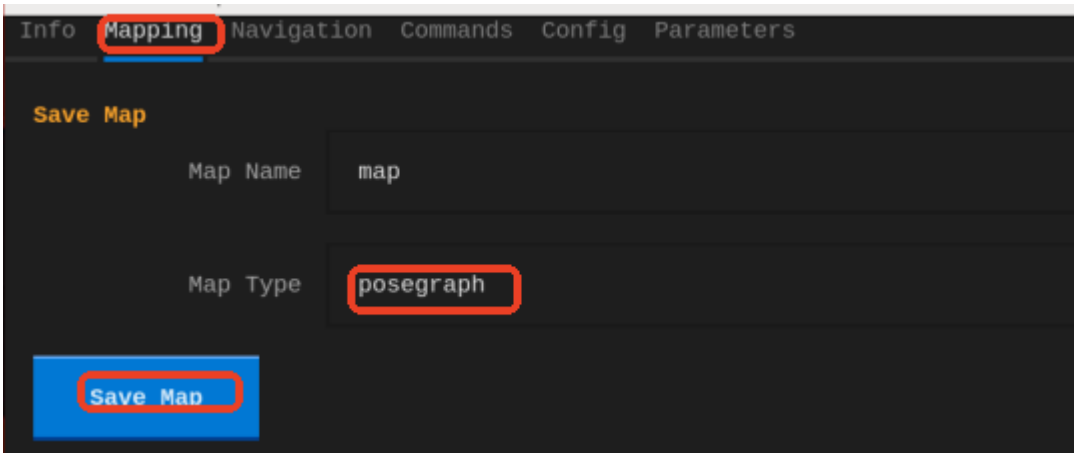
2. Function Introduction

- The Lifelong Mapping feature is implemented using the slam_toolbox algorithm to build a pose map. It is designed to support the robot in continuously building, updating, and maintaining maps in long-term, dynamic environments. Its features are:
  - **Map persistence and reuse**: stores and reloads pose-graph data through serialization and deserialization, supporting continued mapping after an interruption.
  - **Incremental map update**: continues to extend on top of the existing map and updates environmental changes (such as newly added objects and temporary obstacles). This is very important for building maps of large scenes such as warehouses, eliminating the need to start mapping from scratch every time the environment changes.
- Application scenarios
  - For scenarios such as warehouses and shopping malls, the robot needs to perform patrol and transport tasks over the long term. Lifelong mapping can dynamically update the map to cope with changes such as shelf changes and temporary obstacles, while retaining historically valid data and avoiding repeated mapping.
  - When the robot needs to pause due to insufficient battery or a task interruption, after restarting it can continue mapping by loading the existing pose graph, without starting from scratch. For example, a campus inspection robot can complete the construction of a large-area map in multiple time periods.

3. Case Demonstration

- First-time mapping: refer to the previous tutorial [slam-toolbox Algorithm Mapping]

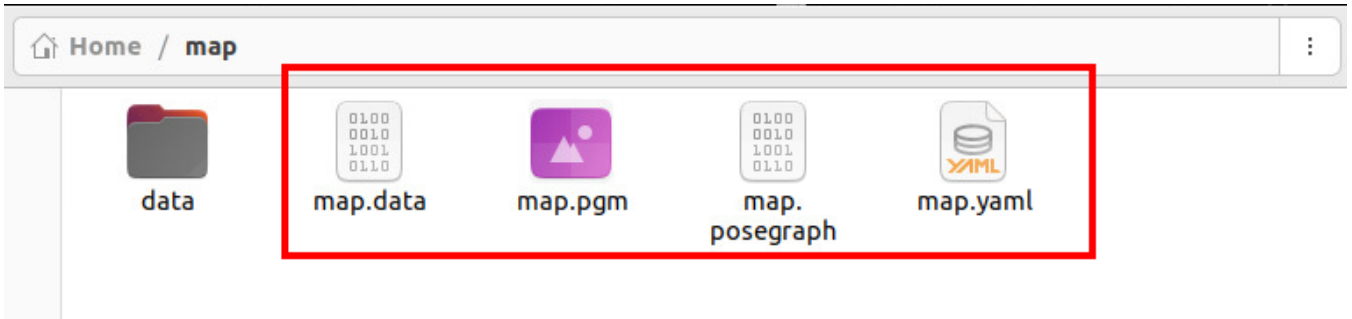
3.1 Saving the Pose Graph



3.2 Saving the Grid Map



- The saved grid map and pose map are in the system's `~/map` directory, where:
  - map.data and map.posegraph are the pose map files
  - map.pgm and map.yaml are the grid map files



- Then press ctrl+c to end mapping

4. Restoring the Mapping State

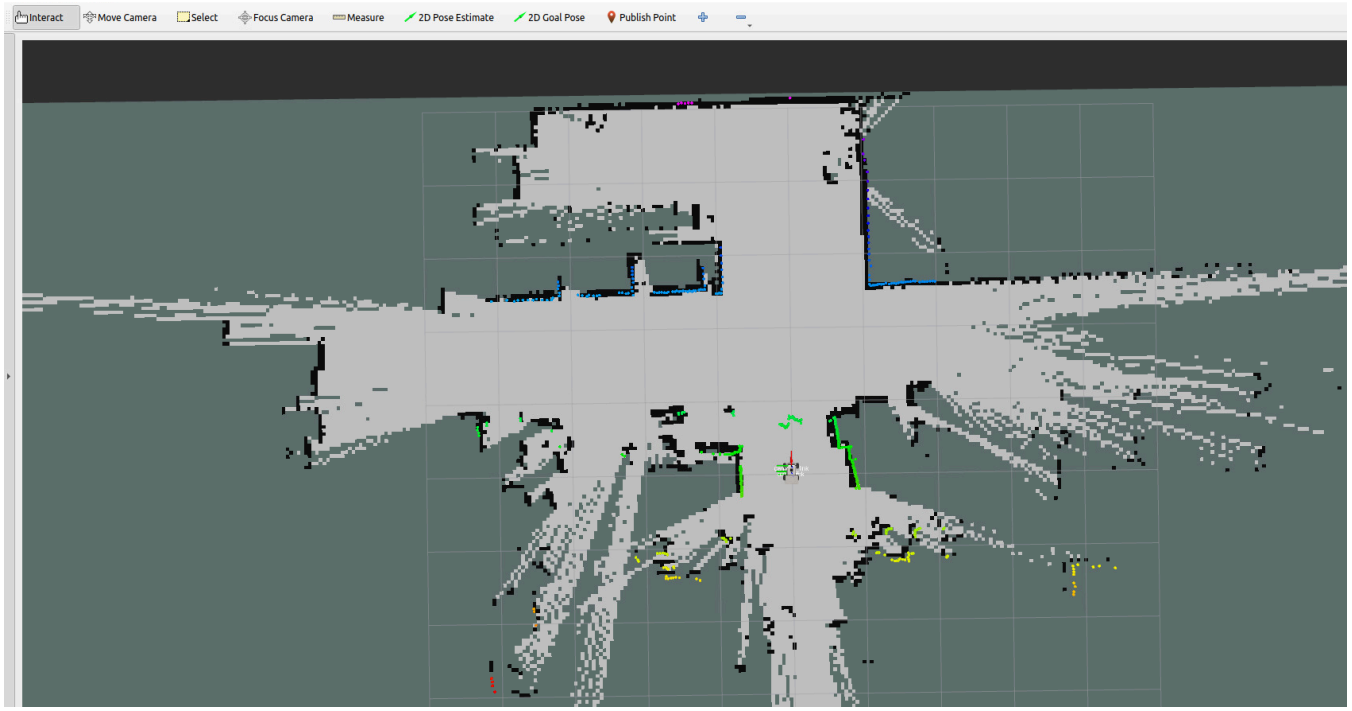
4.1 Restoring from the Mapping Origin (recommended)

- If the car is at the origin of the first mapping, restore the mapping state with the following command:

```
ros2 launch yahboomcar_bringup slam_toolbox.launch.py pose_graph:=map
```

> pose_graph:=map is the name of the pose map passed in

- After that, you can continue mapping or update the map environment from the previously built map state.

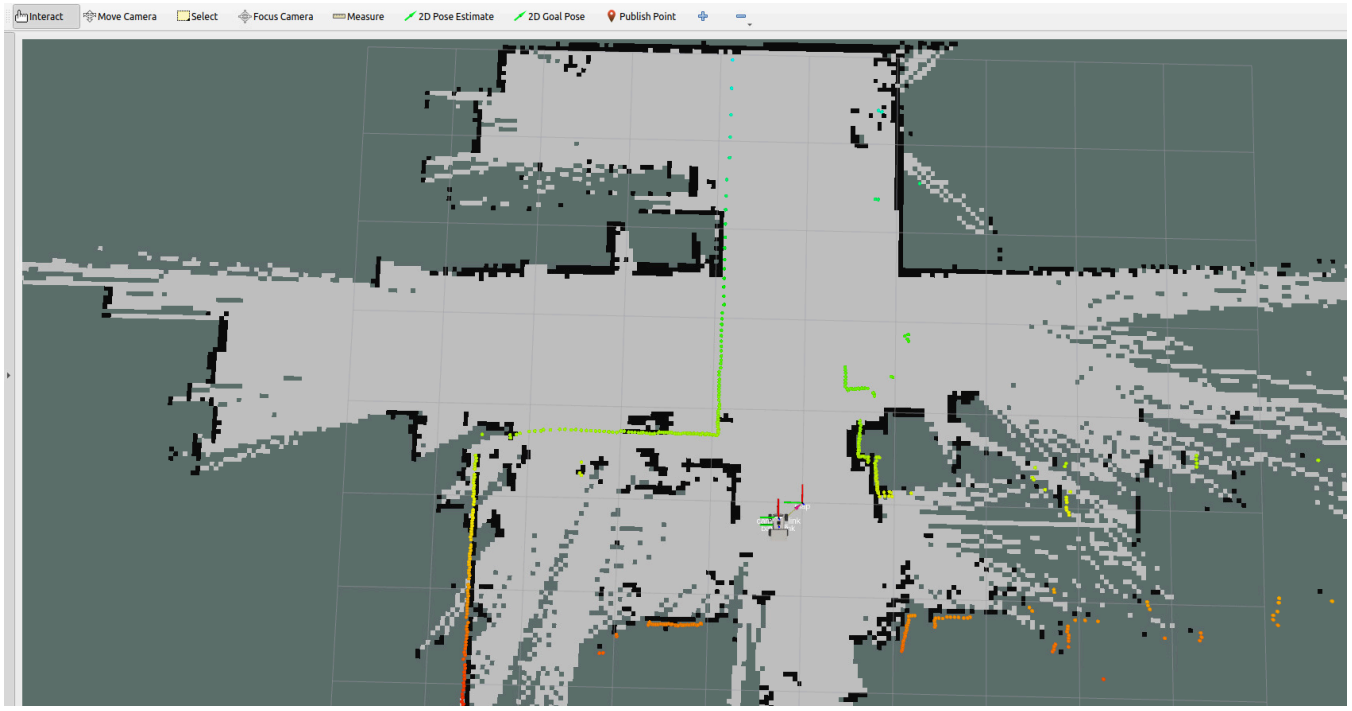


4.2 Restoring from Any Point in the Map

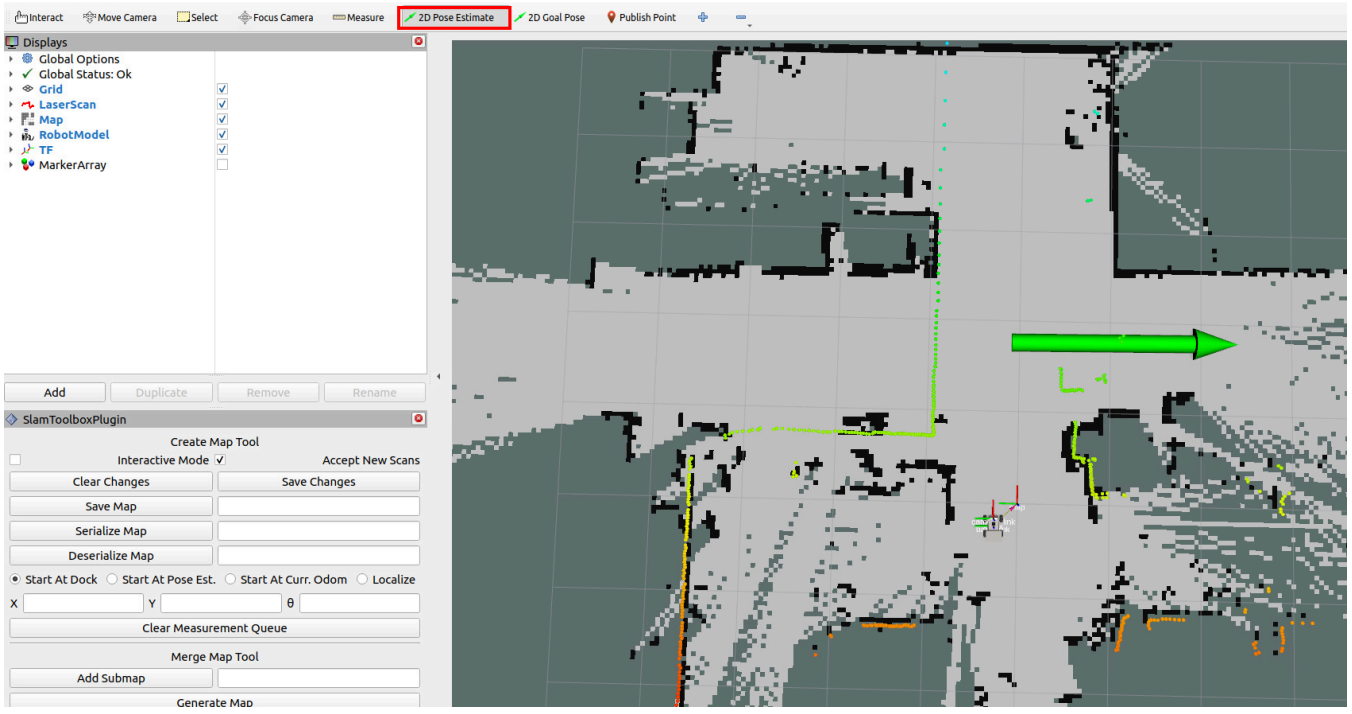
- If the robot is currently at an arbitrary position in the map, first start the mapping node:

```
ros2 launch yahboomcar_bringup slam_toolbox.launch.py pose_graph:=map
```

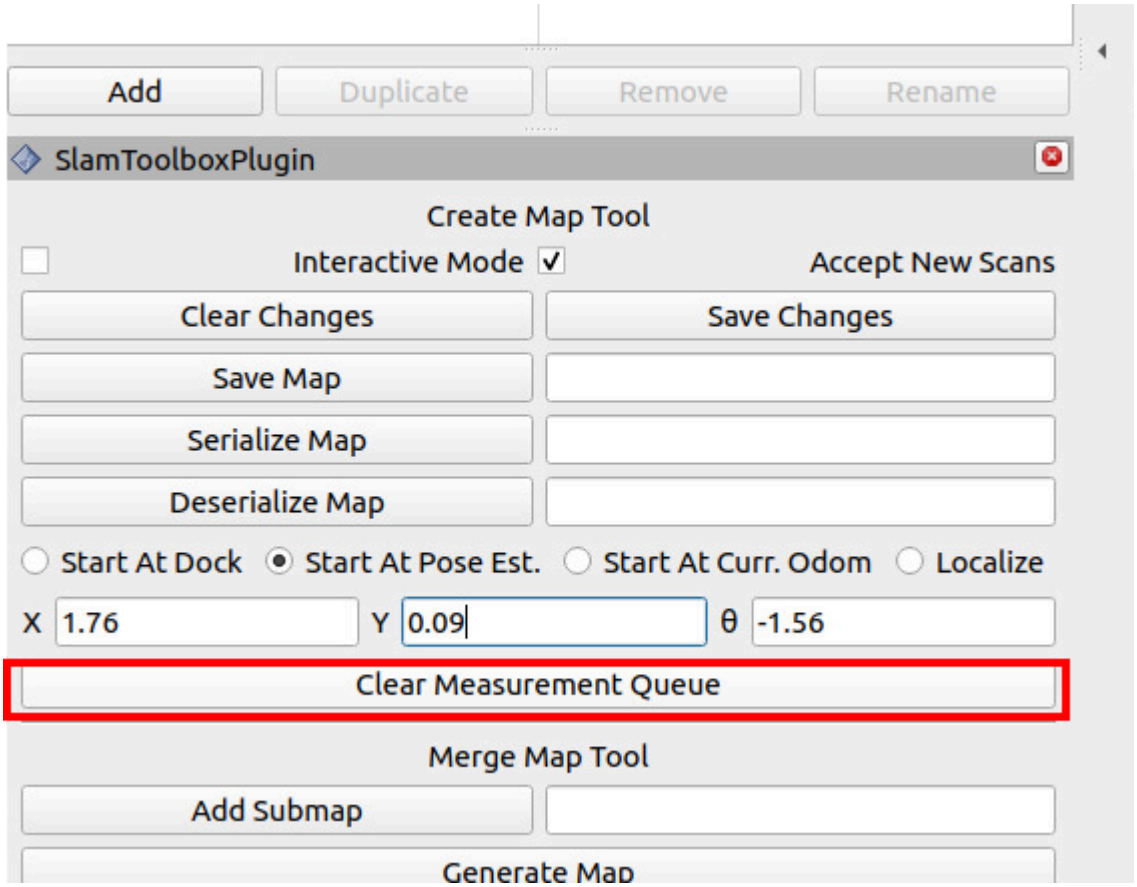
- As can be seen, if the robot is not at the map origin, the laser point cloud and the map's environmental features do not match



- First use the `2D Pose Estimate` tool to give an approximate pose on the map



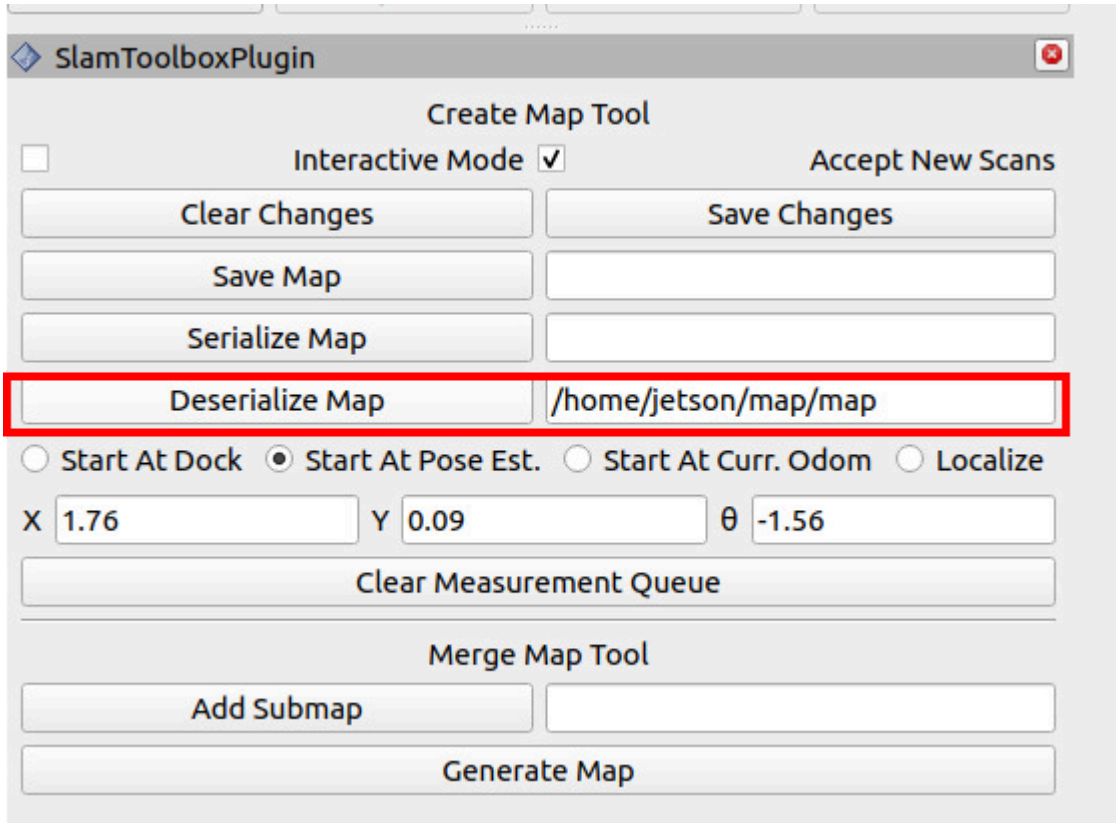
- Click `Clear Measurement Queue` in the left toolbar; the current pose data will then be filled in automatically



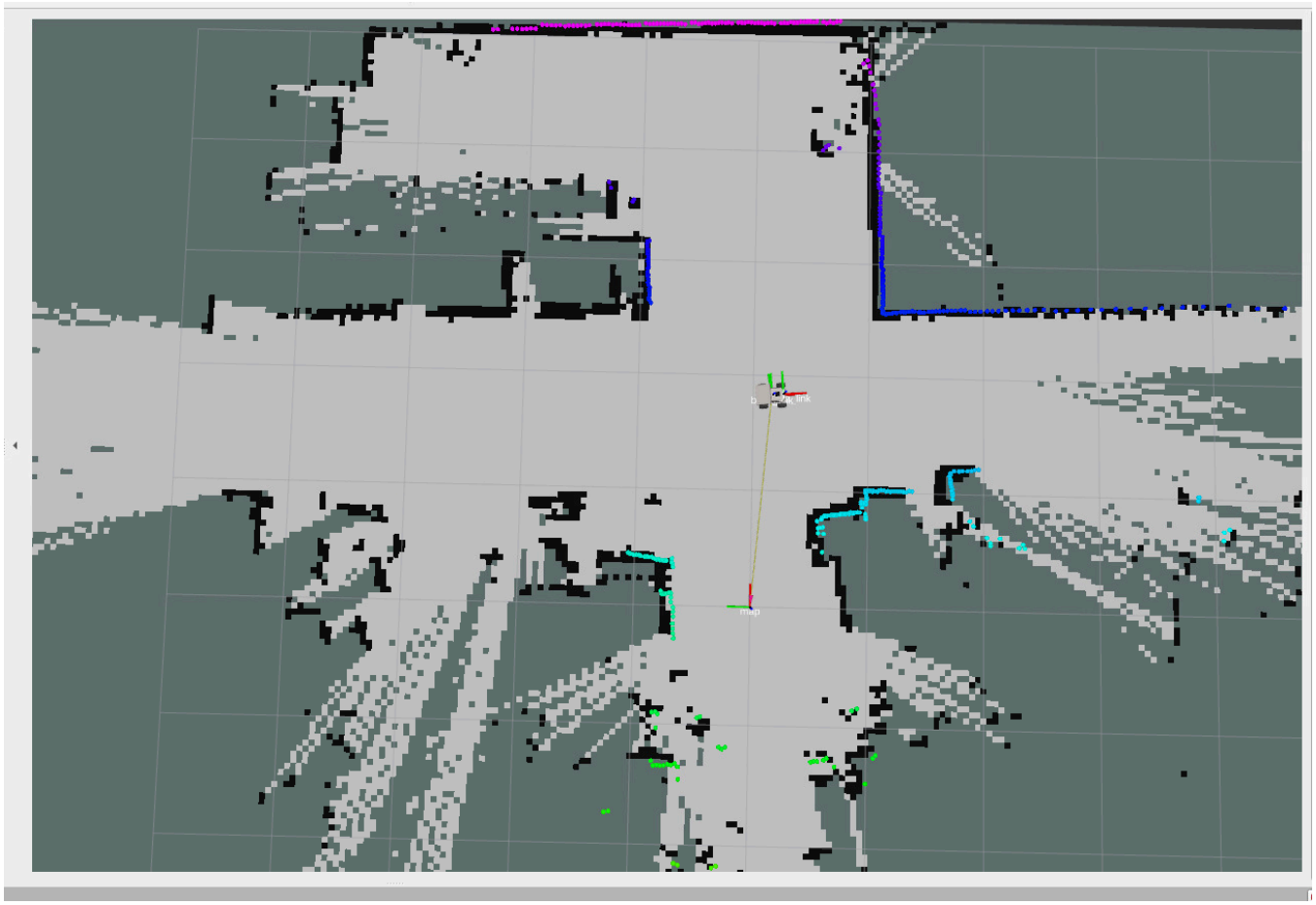
- Then, in the option field after `Deserialize Map`, fill in the path of the previously saved pose map (note that the file extension is not needed)

> **Important**
>
> Note that the `/home/jetson` filled in here is the mainboard's user directory; adjust it according to your mainboard
>
> **OrinNano, Orin Nx** : `/home/jetson`
>
> **RDK X5:** `/home/sunrise`
>
> **Raspberry Pi 5** : `/root/map`

- Then click `Deserialize Map` to deserialize the map.



- The robot has restored its correct pose in the global map, and can then continue mapping or update the map environment from the previously built map state.



4.3 Archiving Mapping Progress

- In addition, you can save pose maps with different names at different stages of mapping progress, just like game saves, to restore the mapping state from different progress points

